



# Lógica para Programação

Projecto

2025-2026

## Um agradável passeio computacional com o teu novo melhor amigo: o Prolog

### Conteúdo

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                          | <b>2</b>  |
| <b>2</b> | <b>O esqueleto do teu programa em Prolog</b>               | <b>2</b>  |
| <b>3</b> | <b>Parte 1: O Prolog e as Bases de Conhecimento</b>        | <b>2</b>  |
| 3.1      | Estruturas de dados . . . . .                              | 3         |
| 3.2      | Questões a responder . . . . .                             | 3         |
| 3.3      | Predicados a implementar . . . . .                         | 4         |
| <b>4</b> | <b>Parte 2: Sai um joguinho (de palavras) em Prolog</b>    | <b>7</b>  |
| 4.1      | Estruturas de dados . . . . .                              | 7         |
| 4.2      | Predicados a implementar . . . . .                         | 8         |
| <b>5</b> | <b>Parte 3: And now for something completely different</b> | <b>10</b> |
| 5.1      | Estruturas de dados e restrições . . . . .                 | 11        |
| 5.2      | Predicados a implementar . . . . .                         | 12        |
| <b>6</b> | <b>Avaliação e entrega</b>                                 | <b>13</b> |
| 6.1      | Cotação . . . . .                                          | 13        |
| 6.2      | Condições de realização e prazos . . . . .                 | 14        |
| 6.3      | Sobre as cópias . . . . .                                  | 14        |
| <b>7</b> | <b>Recomendações</b>                                       | <b>14</b> |

# 1 Introdução

Estás a começar a aprender a programar em Prolog. Na verdade, o Prolog é apenas uma de muitas linguagens de programação que vais estudar ao longo do curso, com todas as suas coisas boas e algumas (pouquíssimas) más. Resolves então ver o que o Prolog tem para te oferecer. Vais exercitar o teu cérebro na arte de programar e darás uns largos passos no caminho de te tornares uma divindade de programação, com uma **enorme capacidade de adaptação** ao que quer que seja que te venha a aparecer pela frente.

Para começar, na Secção 2 podes ver o cabeçalho do teu projecto. Na Secção 3 vais aprender a lidar com bases de dados/conhecimento, na Secção 4 vais implementar um joguinho de palavras em Prolog e, na Secção 5, vais ter de organizar um fim de semana de filmes com os teus amigos. Sobre as condições de realização do projecto, sua avaliação e recomendações, vê, sff, as Secções 6 e 7. Obrigada!

## 2 O esqueleto do teu programa em Prolog

O teu ficheiro em Prolog (extensão .pl) deverá ter as seguintes linhas iniciais:

```
:- encoding(utf8).
% Escreve nesta linha o teu número e nome
:- style_check(-discontiguous). % para nao se queixar descontinuidade da BD
:- set_prolog_flag(answer_write_options,[max_depth(0)]). % ver listas completas
:- ['codigoAuxiliar.pl']. % Ficheiro dado
:- ['bd_estudantes.pl']. % Ficheiro dado
:- ['listas_palavras.pl']. % Ficheiro dado
% O teu código deve começar na próxima linha
```

- Em todo o projecto, caso os resultados sejam numéricos, estes devem ter no máximo duas casas decimais. Usa o predicado `arredonda/2`, dado no ficheiro “`codigoAuxiliar.pl`”;

- Todos os exemplos deste enunciado estão no ficheiro “`testes_publicos.plt`”. Para os correrres, carrega o ficheiro e faz: “`run_tests.`”;

- **ATENÇÃO: não debes copiar NUNCA bocados de texto a partir deste pdf, correndo o risco de ficares com um input mal formatado (problema: muitas vezes não é visível).**

## 3 Parte 1: O Prolog e as Bases de Conhecimento

Uma **base de dados** é um repositório organizado de dados<sup>1</sup> e uma **base de conhecimento** é uma base de dados às quais se associam regras/mecanismos de inferência que permitem “raciocinar” sobre esse conhecimento. Pensando no que já aprendeste do Prolog, um ficheiro de factos é uma base de dados; quando lhe acrescentas regras, passa a ser uma base de conhecimento.

Com esta informação em vista, descobres uma colecção de dados – “`bd_estudantes.pl`” – que relaciona informação sobre uma população de estudantes e o seu desempenho académico<sup>2</sup>. Decides analisar essa colecção e implementar várias funcionalidades em Prolog que consideras pertinentes (e tantas mais seriam possíveis!).

<sup>1</sup>Vais ter uma disciplina chamada “Bases de dados” no segundo ano!

<sup>2</sup>Estes dados foram adaptados de uma coleção do Kaggle (<https://www.kaggle.com>)

### 3.1 Estruturas de dados

Começas por analisar a estrutura de dados<sup>3</sup> do ficheiro “bd\_estudantes.pl”. Este contém um conjunto de factos que caracterizam cada estudante, sendo cada estudante identificado através de um identificador único (Id), com a forma sN em que  $N \in \mathbb{N}$ , que o liga aos vários predicados que o definem:

- `estudante(Id, Idade, Género)`
- `atividade(Id, HorasEstudo, HorasEscrã, FrequênciaAulas)`
- `saude(Id, HorasSono, QualidadeAlimentação, ExercícioFísico, SaúdeMental)`
- `exame(Id, NotaExame)`

Podes assumir que:

- “Género” toma os seguintes valores: feminino, masculino, outro;
- “HorasEstudo”, “HorasEscrã” (em atividades lúdicas) e “HorasSono” indicam a média dos valores diários e tomam valores em  $\mathbb{Q}$ ;
- “FrequênciaAulas” é em percentagem;
- “QualidadeAlimentação” é fraca, razoável ou boa (fraca, razoavel, boa);
- “ExercícioFísico” toma valores em  $\mathbb{N}$ , de 1 a 6;
- “SaúdeMetal” toma valores em  $\mathbb{N}$ , de 1 (pior) a 10 (melhor);
- “NotaExame” toma valores em  $\mathbb{Q}$  (max 100).

Segue-se um trecho de “bd\_estudantes.pl” relativo ao estudante identificado por “s1596”:

```
estudante(s1596, 21, masculino).
atividade(s1596, 3.3, 1.9, 98.2).
saude(s1596, 8.3, razoavel, 3, 1).
exame(s1596, 66.1).
```

### 3.2 Questões a responder

Olhando para a base de dados, decides que gostarias de perceber o seguinte (entre tantas outras hipóteses):

- Qual a média das notas?
- Qual a média dos alunos:
  - que têm X ou menos anos?
  - que têm entre X e Y anos?
  - que têm mais de Y anos?
- Que grupo de estudantes vai mais às aulas (feminino, masculino, outro)?

---

<sup>3</sup>Também vais ter uma disciplina dedicada a estruturas de dados!

- Quais os estudantes que não estão a cuidar da sua saúde?
- Qual é a probabilidade de um estudante passar muitas horas à frente do ecrã e ter boas notas?
- Será que existe alguma correlação:
  - entre os alunos que vão mais às aulas e as notas?
  - entre a qualidade da saúde mental dos alunos e as notas que obtêm?

### 3.3 Predicados a implementar

Sabendo que a média (ou média aritmética) é a soma de todos os valores dividida pelo número total de observações, implementas o predicado genérico `media/2` e o predicado `mediaNotasPorIdade/2` tal que:

- `media(ListaValores, Media)` é verdade se `ListaValores` for a lista com todos os valores observados e `Media` a sua média;
- `mediaNotasPorIdade(IdadeMin, IdadeMax, Media)` é verdade se `Media` for a média dos valores observados para estudantes com idades maiores que `IdadeMin` e iguais ou menores que `IdadeMax`.

Por exemplo,

```
?- media([1, 2, 4, 6], Media).
Media = 3.25.
```

```
?- media([], Media).
Media = 0.
```

```
?- media([1.2, 1.5, 10, 9.6], Media).
Media = 5.57.
```

```
?- mediaNotasPorIdade(16, 18, Media).
Media = 69.87.
```

```
?- mediaNotasPorIdade(16, 30, Media).
Media = 69.95.
```

Dica: se quiseres calcular a média das notas de todos os alunos, basta fazer:

```
?- findall(Nota, exame(_, Nota), ListaNotas), media(ListaNotas, Media).
... (ignorar)
Media = 69.95.
```

De seguida, implementas os predicados `freqPorGenero/2` e `alertaSaude/4`, tal que:

- `freqPorGenero(Genero, MediaFreq)` é verdade se `MediaFreq` for a média dos valores de frequência das aulas dos alunos de género `Genero`;
- `alertaSaude(HorasSono, Exercicio, SaudeMental, ListaAlunos)` é verdade se `ListaAlunos` for a lista ordenada de alunos cuja qualidade de alimentação é “fraca”, que dormem menos de `HorasSono` horas, com uma pontuação de exercício físico abaixo de `Exercicio` e uma pontuação relativa à saúde mental abaixo de `SaudeMental`.

Por exemplo,

```
?- freqPorGenero(feminino, Media).
Media = 84.70.
```

```
?- alertaSaude(6, 3, 5, ListaAlunos).
ListaAlunos = [s1067,s1212,s1348,s1598].
```

```
?- alertaSaude(7, 4, 6, ListaAlunos).
ListaAlunos = [s1033,s1051,s1067,s1070,s1118,s1163,s1164,s1209,
s1212,s1262,s1298,s1348,s1376,s1399,s1472,s1518,s1551,s1598].
```

Segue-se o predicado probEcraNotasAltas/3, tal que:

probEcraNotasAltas(HorasEcra, Nota, Probabilidade) é verdade se Probabilidade é a probabilidade de um aluno ter uma nota acima de Nota tendo em conta que passa HorasEcra horas à frente de um ecrã (em actividades lúdicas). Para tal, utiliza a Equação 1 (neste caso,  $A$  representará o evento de um aluno ter uma nota acima de Nota e  $B$  o evento de um aluno passar mais do que HorasEcra horas à frente de um ecrã).

$$P(A | B) = \frac{\#(A \cap B)}{\#(B)} \quad (1)$$

Por exemplo,

```
?- probEcraNotasAltas(4.3, 80, Probabilidade).
Probabilidade = 0.
```

```
?- probEcraNotasAltas(4, 80, Probabilidade).
Probabilidade = 0.21.
```

Finalmente, atacas a correlação, em particular o Coeficiente de Correlação de Pearson entre duas variáveis  $X$  e  $Y$ <sup>4</sup> (Equação 2), dada pela fórmula:

$$r = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (2)$$

em que  $X = (X_1, X_2, \dots, X_n)$  e  $Y = (Y_1, Y_2, \dots, Y_n)$  são os valores das variáveis,  $X_i$  e  $Y_i$  representam os valores individuais de cada observação, e  $\bar{X}$  e  $\bar{Y}$  são as médias de  $X$  e  $Y$ , respectivamente.

Analisando esta equação, decides partir a fórmula em pedacinhos e implementar os predicados subtraiValorDeLista/3, somaQuadrados/2, produtoEscalar/3 (relaciona estes predicados com a Equação 2) e, finalmente, correlação/3, tais que:

<sup>4</sup>Vais estudar este conceito em Probabilidades e Estatística.

- `subtraiValorDeLista(Lista, Valor, Resultado)` é verdade se `Resultado` for a lista que se obtém depois de tirar o valor `Valor` de todos os elementos da lista `Lista`;
- `somaQuadrados(Lista, Resultado)` é verdade se `Resultado` for a lista que se obtém depois de se somar o quadrado de cada elemento da lista `Lista`.
- `produtoEscalar(Lista1, Lista2, Resultado)` é verdade se `Resultado` for a lista que se obtém depois de se calcular o produto escalar entre os elementos de `Lista1` e `Lista2` (isto é, somar os resultados das multiplicações dos elementos nas mesmas posições nas duas listas). Este predicado deve falhar se as listas `Lista1` e `Lista2` não tiverem o mesmo tamanho.
- `correlacao(Lista1, Lista2, Resultado)` é verdade se `Resultado` for a lista que se obtém depois de se calcular a correlação entre os valores das listas `Lista1` e `Lista2`, de acordo com a Equação 2.

Por exemplo,

```
?- subtraiValorDeLista([2, 7, 8, 1], 4, Resultado).
Resultado = [-2,3,4,-3].
```

```
?- somaQuadrados([2, 7, 8, 1], Resultado).
Resultado = 118.
```

```
?- produtoEscalar([2, 4, 6], [1, 2, 3], Resultado).
Resultado = 28.
```

```
?- correlacao([1, 2, 3], [60, 70, 90], Resultado).
Resultado = 0.98.
```

```
?- correlacao([1, 2, 3], [90, 70, 60], Resultado).
Resultado = -0.98.
```

```
?- correlacao([1, 10, 6], [60, 100, 90], Resultado).
Resultado = 0.98.
```

Correlação entre o número de horas de estudo (HE) e as notas:

```
?- findall(HE, atividade(ID, HE, _, _), ListaHE),
   findall(Nota, exame(ID, Nota), ListaNotas),
   correlacao(ListaHE, ListaNotas, Corr).
... (ignorar)
Corr = 0.8408716019948734. % Uau!
```

Correlação entre a saúde mental (SM) e as notas:

```
?- findall(SM, saude(ID, _, _, SM), ListaSM),
   findall(Nota, exame(ID, Nota), ListaNotas),
   correlacao(ListaSM, ListaNotas, Corr).
... (ignorar)
Corr = 0.3340392640816457. % Oh!
```

Pensas na quantidade de informação que se encontra escondida nas profundezas da base de dados, mas, de repente, apetece-te um jogo. Vamos lá!

## 4 Parte 2: Sai um joguinho (de palavras) em Prolog

Lembras-te de implementar o “Wordaily”, um jogo de palavras em que se tenta adivinhar uma palavra secreta com um número limitado de tentativas, recebendo pistas com base nas letras corretas/incorrectas e nas suas posições. Se não conheces o jogo podes jogar a versão do New York Times<sup>5</sup>, “wordle”, ou instalar a app “Wordaily”. Na Figura 1 tens um exemplo de um jogo (resolvido) no “Wordaily”.



Figura 1: Joguinho resolvido do “Wordaily”. Há que descobrir uma palavra mistério, neste caso com 5 letras. A primeira linha corresponde ao primeiro palpite, a segunda linha ao segundo palpite, etc. Neste caso, as letras a preto não existem na palavra mistério; as letras a amarelo existem, mas estão na posição errada; as letras a verde existem e estão na posição correcta. A palavra mistério é “FORTE” e foi descoberta ao quarto palpite.

Nota que é sempre muito importante perceber bem o objeto de estudo (neste caso, terás mais facilidade em compreender os predicados a implementar se conheceres bem o jogo em causa). Nota que esta é uma dica para a vida, não apenas para LP: tenta sempre saber o máximo possível sobre o problema que tens em mãos.

Neste caso concreto, assumes que:

- O tamanho da palavra a adivinhar pode ser qualquer inteiro  $> 1$ ;
- O número de tentativas possíveis é um inteiro qualquer;
- Vais implementar três tipos de pistas (já verás quais).

### 4.1 Estruturas de dados

Encontras duas listas de palavras em Português, uma grande, a lista “pt”, outra minúscula, a “mini” (boa para testes iniciais) no ficheiro dado “listas\_palavras.pl”. As suas entradas têm a forma que se segue, em que `Id` é uma string que identifica a lista (“pt” ou “mini”) e `Lista` é a lista de palavras em si:

```
lista_palavras(Id, Lista).
```

Decides que cada palavra será convertida numa lista. Sugestão: usa o predicado pré-definido do Prolog `string_chars(String, ListChars)`. Por exemplo:

```
?- string_chars(batata, L).
L = [b,a,t,a,t,a].
```

<sup>5</sup><https://www.nytimes.com/games/wordle/index.html>

## 4.2 Predicados a implementar

Antes de atacar os diferentes tipos de pistas, decides implementar alguns predicados que te podem vir a ser úteis: tamanho/2, verificaECalcula/4, quantasN/3, quantasC/3, apagaElemento/3 e, finalmente, posicoesPalavra/2, tais que:

- tamanho(Palavra, Tamanho) é verdade se Tamanho é o tamanho (em número de caracteres) da palavra Palavra;
- verificaECalcula(Palavra1, Palavra2, CaracteresPalavra1, CaracteresPalavra2) é verdade se Palavra1 e Palavra2 têm o mesmo tamanho (o predicado falha caso contrário), CaracteresPalavra1 é a lista de caracteres da palavra Palavra1 e CaracteresPalavra2 é a lista de caracteres da palavra Palavra2;
- quantasN(Id, N, Quantas) é verdade se Quantas é o número de palavras de tamanho N da lista identificada por Id;
- quantasC(Id, C, Quantas) é verdade se Quantas é o número de palavras começadas pelo carácter C (podes assumir que será sempre apresentado em minúscula ou entre plicas), da lista identificada por Id;
- apagaElemento(Elemento, Lista1, Lista2) é verdade se a lista Lista2 é o resultado de apagar da lista Lista1 a \*\*\*primeira ocorrência\*\*\* do elemento Elemento; se esse elemento não existe na lista Lista1, Lista2 é igual a Lista1;
- posicoesPalavra(Palavra, Posicoes) é verdade se a lista Posicoes é a lista \*\*\*ordenada\*\*\* (usa o sort/2) de pares que associam os diferentes caracteres da palavra Palavra à sua posição.

Por exemplo,

```
?- tamanho(fidalgo, T).
T = 7.
?- tamanho(cão, T).
T = 3.
?- tamanho("Palhaço", T). % vê o acontece sem as aspas. Oops.
T = 7.

?- verificaECalcula(palhaço, fidalgo, Lista1, Lista2).
Lista1 = [p,a,l,h,a,ç,o],
Lista2 = [f,i,d,a,l,g,o].
?- verificaECalcula(palhaço, cão, Lista1, Lista2).
false.

?- quantasN(mini, 7, Quantas).
Quantas = 3. % há 3 palavras de tamanho 7 na lista mini
?- quantasN(mini, 4, Quantas).
Quantas = 0. % há 0 palavras de tamanho 4 na lista mini
?- quantasC(mini, f, Quantas).
Quantas = 1. % há 1 palavras começada por f na lista mini
?- quantasC(pt, f, Quantas).
Quantas = 47.
?- quantasC(mini, 'M', Quantas).
Quantas = 1.

?- apagaElemento(8, [2, 8, 3, 8, 4], L).
L = [2,3,8,4].
?- apagaElemento(5, [2, 8, 3, 8, 4], L).
L = [2,8,3,8,4].
```

?- posicoesPalavra(batata, Posicoes). % há um a nas posições 2, 4 e 6, um b...  
 Posicoes = [(a,2),(a,4),(a,6),(b,1),(t,3),(t,5)].

Finalmente, atacas as pistas, implementando os predicados pista1/3, pista2/3 e pista3/3. A palavra Palavra1 será a palavra mistério e a palavra Palavra2 o teu palpite. Em todos os casos, os predicados devem falhar se Palavra1 e Palavra2 não tiverem o mesmo tamanho; Pista é uma lista com o mesmo tamanho de Palavra1 e Palavra2.

- pista1(Palavra1, Palavra2, Pista) é verdade se Pista tem:
  - 2 na posição i, se Palavra1 e Palavra2 tiverem o mesmo carácter na posição i;
  - 0 na posição i, caso contrário.
- pista2(Palavra1, Palavra2, Pista) é verdade se Pista tem:
  - 2 na posição i, se Palavra1 e Palavra2 tiverem o mesmo carácter na posição i;
  - 1 na posição i, se o carácter i da palavra Palavra2 é diferente do da posição i da palavra Palavra1, mas existe algures na palavra Palavra1;
  - 0 caso contrário.
- pista3(Palavra1, Palavra2, Pista) é verdade se Pista tem:
  - 2 na posição i, se Palavra1 e Palavra2 tiverem o mesmo caracter na posição i;
  - 1 na posição i, se o caractere i da palavra Palavra2 é diferente do da posição i da palavra Palavra1, mas existe algures na palavra Palavra1, mas não na posição correcta. Por outro lado, se em Palavra2 existirem N caracteres C **em posições erradas**, os primeiros M são marcados com 1 se existirem M caracteres C na palavra Palavra1, os quais ainda não descobriste onde estão (**Don't panic: exemplos abaixo!**);
  - 0 na posição i caso contrário.

Para se perceber melhor a diferença entre a pista2 e a pista3, considera que a palavra mistério é a string xayaza e o teu palpite abacaa. Podes ver na Figura 2 este exemplo (ao qual se juntou a pista 1).

| Posições            | 1 | 2 | 3 | 4 | 5 | 6 |
|---------------------|---|---|---|---|---|---|
| Palavra 1: Mistério | x | a | y | a | z | a |
| Palavra 2: Palpite  | a | b | a | c | a | a |
| Pista 1             | 0 | 0 | 0 | 0 | 0 | 2 |
| Pista 2             | 1 | 0 | 1 | 0 | 1 | 2 |
| Pista 3             | 1 | 0 | 1 | 0 | 0 | 2 |

Figura 2: Palavra mistério xayaza e palpite abacaa.

- Tendo em conta a pista2, a pista será [1, 0, 1, 0, 1, 2] porque b e c não existem na palavra xayaza (e daí os 0); o último a de abacaa está na posição correcta (e daí o 2); as restantes letras (todas a) não estão na posição correcta, mas existem na palavra xayaza (e daí os 1).
- Já tendo em conta a pista3, a pista será [1, 0, 1, 0, 0, 2] porque b, e c não existem na palavra xayaza (e daí os 0 na posição 2 e 4); o último a de abacaa está posição correcta (e daí o 2); os 1 anteriores mantêm-se, excepto o da posição 5, onde tens agora um

0, porque existindo no teu palpite 3 a's em posições erradas ( $N = 3$ ) e 2 a na palavra mistério ( $M = 2$ ), os primeiros 2 a's do teu palpite têm um 1 associado e o restante 0.

Mais exemplos,

```
?- pista1(lara, lar, L).
false.
?- pista1(lara, pera, L).
L = [0,0,2,2].
?- pista1(ramal, arara, L).
L = [0,0,0,0,0].
?- pista1(accelera, batatas, L).
L = [0,0,0,0,0,0,0].

?- pista2(lara, lar, L).
false.
?- pista2(lara, pera,L).
L = [0,0,2,2].
?- pista2(ramal, arara, L).
L = [1,1,1,1,1].
?- pista2(accelera, batatas, L).
L = [0,1,0,1,0,1,0].

?- pista3(lara, lar, L).
false.
?- pista3(lara, pera,L).
L = [0,0,2,2].
?- pista3(ramal, arara, L).
L = [1,1,1,0,0].
?- pista3(accelera, batatas, L).
L = [0,1,0,1,0,0,0].
?- pista3(babaa, aabba, L).
L = [1,2,2,1,2].
?- pista3(babaa, aabca, L).
L = [1,2,2,0,2].
```

Entretanto, misteriosamente, descobres o ficheiro “jogar.pl”, com o código que te permite jogar tendo em conta as pistas que implementaste. Jogas um bocadinho, mas depois ficas a pensar que a interface é mesmo uma desgraça e que uma boa interface é mesmo muito importante.<sup>6</sup>

## 5 Parte 3: And now for something completely different

Estás já a pensar no próximo fim de semana. Combinaste com os teus amigos uma maratona de cinema. Decidiram ver filmes recomendados por uns amigos da Gen-X:

- Strange Days (1995) – um dos melhores filmes de sempre, apesar de ninguém saber.
- Léon (1994) – primeiro filme da Natalie Portman.

---

<sup>6</sup>Felizmente, vais ter uma disciplina sobre interfaces. Pensamos em tudo!

- Memento (2000) – um homem que sofre de um tipo de amnésia que não lhe permite adquirir novas memórias, tenta descobrir quem matou a sua mulher. Genial!
- Indiana Jones, os Salteadores da Arca Perdida (1981) – um must para todas as gerações. Se ainda não viste, shame on you!.
- JFK (1991) – sobre o assassinato do presidente Americano; montagem espectacular; óscares.
- Amadeus (1984) – sobre a vida de Mozart, vários óscares, incluindo o de melhor filme. Incrível. Preparar lenços.
- Top Secret (1984) – hilariante (pelo menos nos anos 80).

Os teus colegas continuam a enviar mensagem com restrições:

- Só consigo estar sábado às 14h e gostava de ver o “Léon”;
- Só consigo estar no domingo às 14h e gostava de ver o “Amadeus”;
- Acho que era fixe fecharmos a maratona com o “JFK”;
- Parece que “O Strange Days” e o “Memento” são algo difíceis de digerir e não devem ser seguidos;
- Como o “Top Secret” é uma comédia, se calhar devia ser logo a seguir ao “Amadeus”, para o caso de ficarmos abalados;
- Não vou estar sábado às 17h e não queria que o “Memento” fosse a essa hora porque gostaria de o ver.

Ficas a pensar que o melhor é implementar algo genérico, para ser possível organizar outros fins de semana de maratonas cinéfilas. No entanto, para te facilitar o trabalho, decides assumir que:

- Serão sempre, no máximo, 7 filmes, 4 a ver no sábado e 3 no domingo;
- No sábado, as sessões serão: pelas 14h (sessão 1), pelas 17h (sessão 2), pelas 20h (sessão 3) e pelas 23h (sessão 4);
- No domingo, as sessões serão: pelas 14h (sessão 5), pelas 17h (sessão 6) e pelas 20h (sessão 7)<sup>7</sup>;
- Consideras o filme “empty” para cada sessão que fiquem vazias, se te forem dados menos de 7 filmes;
- Os filmes de terror só podem ser vistos a partir das 20h (incluindo a sessão das 20h).

## 5.1 Estruturas de dados e restrições

Consideras que os filmes a ver são dados numa lista com um tamanho máximo de 7 e que o escalonamento é uma lista de listas em que cada lista tem uma hipótese de ordem (a posição  $i$  corresponde à sessão  $i$  – ou seja, as sessões 1, 2, 3 e 4 ocupam as quatro primeiras posições da lista; as sessões 5, 6 e 7 ocupam as três últimas posições da lista), tendo em conta as restrições. Considerando estas últimas, tens em conta as mensagens dos teus colegas e mais algumas de que te lembrás, e consideras que as restrições possíveis são dadas por:

---

<sup>7</sup>Como são jovens responsáveis, com aulas no dia seguinte e não haverá sessão às 23h

- `terror(Filme)`: é verdade se Filme for um filme de terror;
- `soPode(Filme, Sessao)`: é verdade se o filme Filme só puder ser visto na sessão Sessao;
- `nunca(Filme, Sessao)`: é verdade se o filme Filme não puder ser visto na sessão Sessao;
- `seguido(Filme1, Filme2)`: é verdade se o filme Filme2 deve ser visto, no mesmo dia, logo a seguir ao filme Filme1;
- `naoSeguido(Filme1, Filme2)`: é verdade se os filmes Filme1 e Filme2 não devem ser vistos, no mesmo dia, um a seguir ao outro (nem Filme1 logo a seguir a Filme2 nem Filme2 logo a seguir a Filme1);
- `antes(Filme1, Filme2)`: é verdade se o filme Filme1 deve ser visto antes do Filme2, mas não necessariamente imediatamente antes.

## 5.2 Predicados a implementar

Decides implementar o predicado `maratonaFilmes/3`, tal que:

`maratonaFilmes(ListaFilmes, ListaRestricoes, Programacao)` é verdade se ListaFilmes for a lista com o nome dos filmes que vão ver, ListaRestricoes for a lista de restrições a verificar (ver acima) e Programacao for uma lista ordenada, sem elementos repetidos, com listas que representam as possíveis sequências dos filmes a ver (portanto, pode haver mais do que uma solução; ou, até, não haver nenhuma), tendo em conta as restrições de ListaRestricoes.

Assim, por exemplo<sup>8</sup>,

% Caso 1: o do enunciado

```
?- maratonaFilmes([strangeDays, león, memento, indianaJones, jfk, amadeus, topSecret],
    [soPode(león, 1), soPode(amadeus, 5), soPode(jfk, 7), naoSeguido(memento, strangeDays),
    seguido(amadeus, topSecret), nunca(memento, 2)], Programacao).
Programacao = [[león, strangeDays, indianaJones, memento, amadeus, topSecret, jfk]].
```

% Caso 2: são dados menos do que sete filmes

```
?- maratonaFilmes([f1, f2, f3, f4, f5], [soPode(f1, 1), seguido(f1, f2), terror(f3),
    soPode(f4, 7), naoSeguido(f4, f3), nunca(f5, 6)], Programacao).
Programacao = [[f1, f2, empty, f3, f5, empty, f4], [f1, f2, f3, empty, f5, empty, f4],
    [f1, f2, f3, f5, empty, empty, f4], [f1, f2, f5, f3, empty, empty, f4]].
```

% Caso3: sem solução

```
?- maratonaFilmes([f1, f2, f3, f4, f5], [soPode(f1, 1), seguido(f2, f1)], Programacao).
Programacao = [].
```

Lembras-te que com o Prolog consegues gerar e testar soluções. Tens é de garantir que não explode com a memória. Pensas que se calhar vais ter de usar functores. Ou... não. Hum...

Quando acabas, envias a solução para o próximo fim de semana para os teus amigos. Ficam tão impressionados que te dispensam de pagar as bebidas e as pipocas. Que contribuição épica! E foi só o início.

---

<sup>8</sup>Podes assumir que os nomes dos filmes começam sempre com minúsculas e se tiverem mais do que uma palavra, estas estão concatenadas.

## 6 Avaliação e entrega

### 6.1 Cotação

A nota do projecto será baseada no seguinte:

- Execução correcta – 18 valores distribuídos da seguinte forma:
  1. Parte 1: O Prolog e as bases de conhecimento (7.0 valores)
    - (a) `media/2`: 0.5 valores;
    - (b) `mediaNotasPorIdade/3`: 0.5 valores;
    - (c) `freqPorGenero/2`: 0.75 valores;
    - (d) `alertaSaude/4`: 0.75 valores;
    - (e) `probEcraNotasAltas/3`: 1.5 valores
    - (f) `subtraiValorDeLista/3`: 0.5 valores;
    - (g) `somaQuadrados/2`: 0.5 valores;
    - (h) `produtoEscalar/3`: 0.5 valores;
    - (i) `correlacao/3`: 1.5 valores.
  2. Parte 2: Jogo de palavras (9.0)
    - (a) `tamanho/2`: 0.5 valores;
    - (b) `verificaECalcula/4`: 0.5 valores;
    - (c) `quantasN/3`: 0.5 valores;
    - (d) `quantasC/3`: 0.5 valores;
    - (e) `apagaElemento/3`: 0.5 valores;
    - (f) `posicoesPalavra/2`: 1.5 valores;
    - (g) `pista1/3`: 1.5 valores;
    - (h) `pista2/3`: 1.5 valores;
    - (i) `pista3/3`: 2 valores.
  3. Parte 3: And now for something completely different (2.0 valores)
    - (a) `maratonaFilmes/3`: 2.0 valores.
- Estilo de programação e facilidade de leitura – 2 valores assim distribuídos:
  - Implementação de predicados não excessivamente longos (1.0 valor): uma má abstracção procedimental e duplicações de código serão penalizados. No entanto, o facto de usar um predicado recursivo, desde que bem feito, não é penalizado em relação ao uso de predicados funcionais;
  - Boa escolha de nomes dos predicados auxiliares e das variáveis (1.0 valor): os nomes dos predicados e variáveis deverão tornar o programa mais compreensível.

Presença de *warnings* serão penalizadas (-2 valores). Ausência de comentários ou comentários pobres (por exemplo, cópias das descrições do enunciado) serão também penalizados (respectivamente -1.5 valores e -0.5 valores). Dado que não devemos ter de fazer *scroll* para ler as linhas do vosso código, cada linha deverá ter um máximo de 90 caracteres (penalização de -0.5 se os excederem).

## 6.2 Condições de realização e prazos

O projecto é individual. A entrega final terá lugar às **23h59 de dia 7 de janeiro de 2026**, através do sistema Mooshak (e será usado o SWI Prolog). Depois desta hora, não serão aceites projectos sob pretexto algum<sup>9</sup>. A ter em conta:

- Cada aluno deverá submeter um ficheiro .pl contendo o seu código. As primeiras linhas deverão ser as indicadas na Secção 2;
- A avaliação da execução do código do projecto será feita automaticamente através do sistema Mooshak, usando vários testes configurados no sistema. O tempo de execução de cada teste está limitado, bem como a memória utilizada. Tipicamente, só se poderá efectuar uma nova submissão 15 minutos depois da submissão anterior<sup>10</sup>. Só são permitidas 10 submissões em simultâneo no sistema, pelo que uma submissão poderá ser recusada se este limite for excedido. Nesse caso tentar mais tarde.

Serão publicadas na página da cadeira as instruções necessárias para a submissão do código no Mooshak e a partir dessa altura será possível a submissão por via electrónica. Até ao prazo de entrega poderá efectuar até 35 submissões (não usar o Mooshak para debug), sendo utilizada para efeitos de avaliação a última entrega efectuada; a partir da 35 submissão, terá uma penalização de 0.1 (em 20) por cada submissão efectuada.

Pode ou não haver uma discussão oral do projecto e/ou uma demonstração do funcionamento do programa (será decidido caso a caso).

## 6.3 Sobre as cópias

Os alunos deverão fazer o seu próprio código e deverão saber explicá-lo na totalidade. Deverão também **obrigatoriamente** preencher um formulário (detalhes em breve) a declarar que:

- não usaram o ChatGPT ou outro *Large Language Model* (LLM) qualquer; ou
- usaram e, neste caso, devem explicar como o usaram e como decorreram as interações relativas a CADA predicado.

Caso dois alunos tenham usado um LLM, não o indicarem, e os projectos forem considerados muito semelhantes, serão considerados cópia. Caso um aluno seja convocado para oral e não seja capaz de explicar o seu código, este será considerado copiado. Projectos considerados cópia levarão à reprovação na disciplina e levantamento de processo disciplinar. O corpo docente da disciplina será o único juiz do que se considera uma cópia.

## 7 Recomendações

Durante o desenvolvimento do programa não se esqueçam da Lei de Murphy: a) Todos os problemas são mais difíceis do que parecem; b) Tudo demora mais tempo do que pensamos; c) Se alguma coisa puder correr mal, vai correr mal, na pior altura possível.

<sup>9</sup>Note que o limite de 10 submissões simultâneas no sistema Mooshak implica que, caso haja um número elevado de tentativas de submissão sobre o prazo de entrega, alguns alunos poderão ver-se impossibilitados de submeter o código dentro do prazo.

<sup>10</sup>De notar que, se for feita uma submissão no Mooshak a menos de 15 minutos do prazo de entrega, o aluno fica impossibilitado de efectuar qualquer outra submissão posterior.